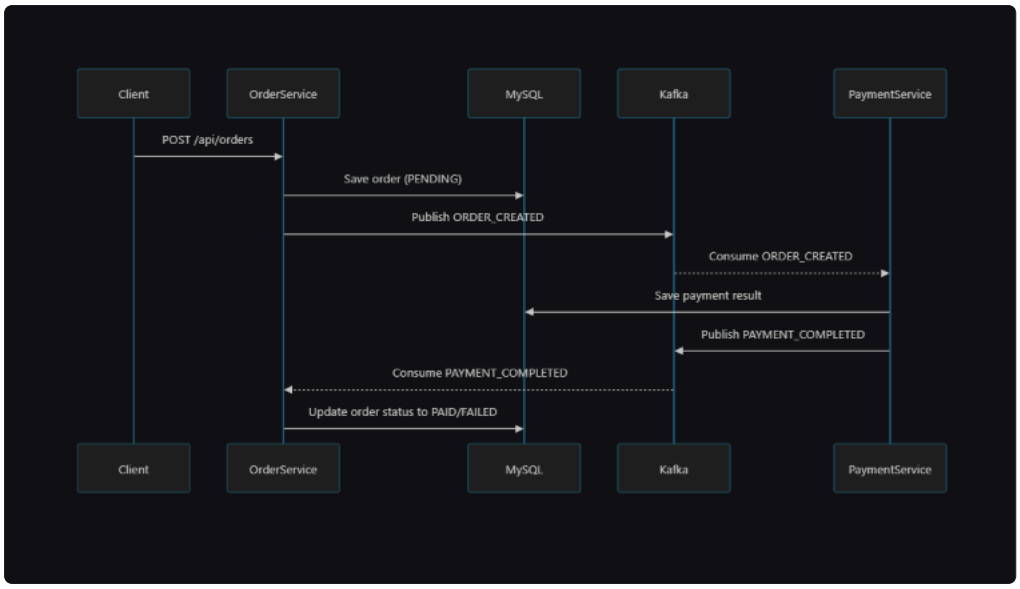


Event-Driven Payment Processing System - Interview Notes

1. Project Overview

This project is a Spring Boot-based event-driven microservices application that simulates an o

The main idea is:
- The order-service accepts a new order
- It stores the order in its own database
- It publishes an Order Created event to Kafka
- The payment-service listens to that event
- It simulates payment processing
- It saves payment information in its own database
- It publishes a Payment Completed event
- The order-service listens to that event and updates the order
 status to PAID or FAILED

This means the system is asynchronous, decoupled, and scalable.

---

2. Why This Project is Important

This project demonstrates many interview-relevant concepts:
- Microservices architecture
- Event-driven communication
- Kafka producer/consumer pattern
- Database per service
- REST APIs
- Asynchronous processing
- Message-based integration
- Distributed system concepts

It is a very strong example of how services communicate without
tightly coupling to each other.

---

3. Project Architecture

High-Level Architecture Diagram

```mermaid
flowchart LR
    Client([Client / Postman])
    OS[order-service :8085]
    PS[payment-service :8086]
    K{{Kafka}}
    MO[(mysql-order)]
    MP[(mysql-payment)]

    Client -->|POST /api/orders| OS
    OS --> MO
    OS -->|order-created-events| K
    K -->|consume| PS
    PS --> MP
    PS -->|payment-completed-events| K
    K -->|consume| OS
    OS -->|update status PAID/FAILED| MO
    Client -->|GET payment| PS
```

Workflow Explanation

1. Client sends a POST request to order-service
2. Order is saved in the order database with status PENDING
3. order-service sends ORDER_CREATED event to Kafka topic
`order-created-events`
4. payment-service consumes that event
5. payment-service simulates payment processing
6. Payment result is saved in payment database
7. payment-service sends PAYMENT_COMPLETED event to Kafka topic
`payment-completed-events`
8. order-service consumes that event and updates order status to
 PAID or FAILED

---

4. Service Responsibilities

A. order-service

This service is responsible for the order domain.

Responsibilities:
- Accept order requests via REST API
- Validate order request
- Save order in MySQL
- Publish order events to Kafka
- Consume payment status events
- Update order status based on payment result

Main classes:
- `OrderController` → exposes REST endpoints
- `OrderService` → business logic
- `OrderRepository` → database access
- `Order` entity → order table mapping

- `OrderEventProducer` → Kafka producer
- `PaymentEventConsumer` → Kafka consumer for payment result

Endpoints:
- POST /api/orders
- GET /api/orders
- GET /api/orders/{id}
- PUT /api/orders/{id}/status

Order statuses:
- PENDING
- PAYMENT_PROCESSING
- PAID
- FAILED

B. payment-service

This service is responsible for payment processing.

Responsibilities:
- Listen for order-created events from Kafka
- Simulate payment processing
- Save payment record in MySQL
- Decide success/failure based on amount
- Publish payment completed event back to Kafka
- Expose APIs to view payment data

Main classes:
- `PaymentController` → exposes payment endpoints
- `PaymentService` → payment processing logic
- `PaymentRepository` → payment database access
- `Payment` entity → payment table mapping
- `OrderEventConsumer` → consumes order-created events
- `PaymentEventProducer` → produces payment result event

Endpoints:
- GET /api/payments
- GET /api/payments/order/{orderId}

Payment statuses:
- PROCESSING
- SUCCESS
- FAILED

---

5. Kafka Flow in Detail

There are two Kafka topics:

1. order-created-events
- Produced by: order-service
- Consumed by: payment-service
- Purpose: notify that a new order has been created

2. payment-completed-events

- Produced by: payment-service
- Consumed by: order-service
- Purpose: notify that payment for an order has completed

Why this is important:
Kafka enables asynchronous and decoupled communication. The order-service does not call the pa

This is the foundation of event-driven architecture.

---

6. Important Java Classes and Their Roles

order-service

OrderController
This controller handles REST requests for order operations.

Example behavior:
- create order
- fetch all orders
- fetch one order by ID
- update status manually

This is the external entry point for orders.

OrderService
This class contains the core business logic.

Whenever a new order is created:
- It converts request into entity
- Saves order in DB
- Creates `OrderEvent`
- Publishes event using Kafka

Important logic:
- `createOrder()` creates and saves an order
- `getOrderById()` fetches one order
- `getAllOrders()` fetches all orders
- `updateOrderStatus()` updates order status

OrderEventProducer
This class publishes messages to Kafka using `KafkaTemplate`.

It sends:
- key = orderId
- value = `OrderEvent`
- topic = `order-created-events`

This is the producer side of the order flow.

PaymentEventConsumer

This class listens for payment completion messages

It does:
- receive `PaymentEvent`
- map payment status to order status
- call `orderService.updateOrderStatus()`

If payment status is success, the order becomes PAID.
If payment fails, the order becomes FAILED.

---

payment-service

OrderEventConsumer
This class listens for `ORDER_CREATED` messages from Kafka.

When an order event is received, it calls:
- `paymentService.processPayment(event)`

This acts as the starting point of payment processing.

PaymentService
This is the most important class in payment-service.

It does:
- checks whether payment already exists for an order
- simulates payment delay via thread sleep
- decides payment success/failure using amount
- saves payment in database
- creates `PaymentEvent`
- sends the result to Kafka

Important method:
- `processPayment(OrderEvent orderEvent)`

It prevents duplicate payments by checking if payment already
exists for that order ID.

PaymentEventProducer
This class publishes payment results to Kafka with the topic:
- `payment-completed-events`

It sends the final payment status back to the order service.

---

7. Database Design

Order Database
The order-service uses an `orders` table.

Entity: `Order`

````
Fields:
- id
- customerName
- productName
- quantity
- amount
- status
- createdAt
- updatedAt

This table represents the lifecycle of the order.

---

### Payment Database
The payment-service uses a `payments` table.

Entity: `Payment`

Fields:
- id
- orderId
- customerName
- amount
- status
- message
- processedAt

The `orderId` is unique to ensure one payment per order.

---

## 8. Entity Status Flow

### Order status flow

```mermaid
stateDiagram-v2
    [*] --> PENDING
    PENDING --> PAYMENT_PROCESSING
    PAYMENT_PROCESSING --> PAID
    PAYMENT_PROCESSING --> FAILED
    PAID --> [*]
    FAILED --> [*]
```

### Payment status flow

```mermaid
stateDiagram-v2
    [*] --> PROCESSING
    PROCESSING --> SUCCESS
    PROCESSING --> FAILED
    SUCCESS --> [*]
    FAILED --> [*]
```

This shows how the system tracks both order and payment lifecycle
````

separately.

---

9. Why Use Separate Databases?

This is a very important microservice principle.

Each service owns its own data:
- order-service owns order data
- payment-service owns payment data

Benefits:
- Loose coupling
- Independent evolution
- Better domain separation
- Simpler service boundaries

Drawbacks:
- Eventual consistency
- More complex debugging
- distributed transactions are harder

---

10. Why Event-Driven Architecture?

Advantages:
- Services are independent
- Payment processing can happen asynchronously
- Order-service can continue processing without waiting for payment
- Better scalability
- Good for real-world enterprise systems

Disadvantages:
- Harder to debug than synchronous calls
- Event ordering and retries need attention
- Need monitoring and logging

---

11. Tech Stack Used

Component	Used For
Java 17	Main backend language
Spring Boot 3.2.5	Application framework
Spring Web	REST controller APIs
Spring Data JPA	Database access
MySQL	Data storage
Kafka	Event-driven communication
Docker Compose	Container orchestration
Maven	Build tool
Lombok	Boilerplate reduction

---

12. Important Configurations

application.yml in order-service
This file configures:
- server port 8085
- MySQL datasource
- JPA/Hibernate
- Kafka bootstrap servers
- consumer group id
- producer/consumer serializers
- Kafka topic names

application.yml in payment-service
This file configures:
- server port 8086
- MySQL datasource
- JPA/Hibernate
- Kafka bootstrap servers
- payment simulation delay
- failure threshold logic

---

13. Payment Logic Details

Inside `PaymentService.processPayment()`:

- If the payment already exists for this order, it skips reprocessing
- It waits for a short simulation delay
- It decides the status based on amount
- If amount <= 0, then payment fails
- It saves payment in database
- It creates a `PaymentEvent`
- It publishes the event to Kafka

This makes it look like a real payment engine but still simple enough for demo purposes.

---

14. What Happens in a Typical Request

Example request
```json
{
  "customerName": "Alice",
  "productName": "Laptop",
  "quantity": 1,
  "amount": 999.99
}
```

Execution
- order is created and saved in `orders` table
- status starts as `PENDING`
- `ORDER_CREATED` event is sent
- payment-service receives event and processes payment
- payment succeeds if amount is valid
- app emits `PAYMENT_COMPLETED` event
- order-service updates order to `PAID`

---

15. Why This Is a Good Interview Project

This project shows:
- real-world event-driven design
- asynchronous communication
- service boundaries
- practical use of Kafka
- MySQL integration
- business flow modeling
- microservice principles in action

If an interviewer asks, “Can you explain a microservices architecture you built?” this project

---

16. Interview Answer - Short Version

> This project is an event-driven microservices application
built with Java and Spring Boot. The order-service receives
orders, stores them in MySQL, and publishes an order-created
event to Kafka. The payment-service consumes that event,
simulates payment processing, stores the payment in a separate
 MySQL database, and publishes a payment-completed event. The
 order-service then listens for that event and updates the order
 status to PAID or FAILED. This architecture demonstrates
 asynchronous communication, decoupling, and microservice design
 using Kafka and separate databases.

---

17. Interview Answer - Medium Version

> This project demonstrates an event-driven microservices flow
in a simple but practical way. The order-service acts as the
front-end service for order creation. It validates the request,
 saves the order, and emits an event to Kafka. The payment-service
 is a separate service with its own database and its own
responsibility: processing payments. It listens for the
 order-created event, simulates payment verification, and then
emits a payment result event. The order-service listens to that
 payment result and updates the order state. The use of Kafka
removes direct dependency between services, and separate
databases respect the boundary of each microservice. This is
a classic asynchronous architecture that supports scalability
and service independence, although it introduces eventual
 consistency and operational complexity.

---

18. Common Interview Questions and Answers

Q1: What is this project about?
A: It is an order-payment processing system using microservices
 and Kafka for asynchronous communication.

Q2: Why use Kafka?
A: To decouple services and allow asynchronous communication between order and payment processing.

Q3: Why use separate databases?
A: Because each microservice should own its own data and avoid tight coupling.

Q4: What is the main advantage of event-driven architecture?
A: Services can operate independently and scale without waiting for each other.

Q5: What is the main challenge of this architecture?
A: Eventual consistency and challenge of debugging distributed flows.

Q6: Why is payment processing asynchronous?
A: Real payment systems may take time and should not block the order flow.

Q7: What is the role of order-service?
A: Order creation, persistence, event publication, and final status update.

Q8: What is the role of payment-service?
A: Consume orders, process payments, save records, and publish results.

Q9: What is the difference between REST and Kafka here?
A: REST handles synchronous client-service communication. Kafka handles asynchronous service-to-service communication.

Q10: What do you learn from this project?
A: Microservices, event-driven design, Kafka, database isolation, handling distributed workflows, and asynchronous processing.

---

19. Final Summary

This project is not just a payment app; it is a demonstration of how real-world systems use microservices and messaging to perform business workflows asynchronously.

The core concepts are:
- service separation
- event communication
- database ownership
- asynchronous processing
- distributed system design

That is exactly the kind of architecture interviewers want to hear about when they ask about microservices, Kafka, and event-driven systems.

---

20. Quick Project Explanation in 3 Lines

- order-service creates and stores orders and publishes events
- payment-service consumes those events, processes the payment, and publishes result events
- Kafka connects the two services asynchronously and keeps them decoupled

---

21. Good Final Statement for Interview

> This is a classic event-driven microservices project. The order-service owns order data and publishes order-created events. The payment-service consumes those events, handles payment logic, and sends payment-completed events back. Kafka acts as the communication layer between the two services, which allows asynchronous processing and reduces tight coupling. Each service has its own database, which is a standard and recommended pattern in microservice architecture.

---

22. Simple Flow Summary

```text
Client -> Order Service -> Save Order -> Publish Order Event
                                        |
                                        v
                                Kafka Topic: order-created-events
                                        |
                                        v
                                Payment Service -> Process Payment
                                        |
                                        v
                                Publish Payment Result Event
                                        |
                                        v
                                Kafka Topic: payment-completed-events
                                        |
                                        v
                                Order Service -> Update Order Status
```

This is the full conceptual flow of the project.